Late Waiver Request — How It Works

A plain-English guide to the Late Waiver Request feature in the Employee Attendance mobile app.

---

1. What is a Late Waiver Request?

When an employee checks in after the configured shift-start time plus the late-threshold grace period, the app records the arrival as "late" and computes a deduction against their pay (either a fixed amount or a tiered slab based on minutes late). A **Late Waiver Request** is the employee's formal way of saying *"the lateness was genuine — please don't deduct"* — for example, a train delay, a sick child, a road closure.

The request is reviewed by their manager. If approved, the deduction is reversed; if rejected, the deduction stands.

What it produces:

- One waiver record per request (`hr.attendance.waiver` or similar custom model) linked to a specific `hr.attendance` row.
- A state machine: **Pending → Approved / Rejected**.
- A reverse-deduction entry on approval (server-side accounting hook).

---

2. The daily flow

1. Employee checks in late one morning (say, 09:18 — past the 08:00 + 15 min threshold).
2. The system computes a deduction based on how late they were and the deduction mode (fixed or slabbed).
3. The employee opens the app, navigates to **Late Waiver Request** from the mode picker.
4. The form shows a dropdown of *eligible late attendance records* — every late arrival in a recent window that doesn't already have a waiver against it.
5. They pick today's record. The screen shows the date, lateness in minutes, and the deduction amount.
6. They type a reason (e.g. *"Train delayed by 20 minutes due to signal failure at City Junction"*).

7. They tap **Submit**.
8. The manager gets a notification. They decide.
9. The history tab on the same screen shows the outcome.

---

3. Eligible late attendances — what shows up in the dropdown

Not every late check-in is waiver-eligible. The dropdown only includes records that satisfy ALL of these:

- The check-in was actually flagged as late (after `office_start_hour + late_threshold_minutes`).
- A deduction was computed (so a free / grace-covered late doesn't show up — there's nothing to waive).
- No prior waiver request exists for this `hr.attendance.id` from this employee, OR the prior request was Rejected (re-applying is allowed in some deployments; cleaner deployments lock the record after the first request).
- The check-in is within a recent window — typically the current month or the last 30 days. Older lates can't be waived to keep the audit trail tight.

The dropdown is fetched via `getEligibleLateAttendances(empId)` and refreshed every 4 seconds while the user is on the Waiver tab so background-sync events (e.g. an offline check-in just synced) surface immediately.

---

4. The form

The Late Waiver Request form has:

- **Attendance dropdown** — required. Lists eligible late records with: date, time of check-in, lateness in minutes, deduction amount. The dropdown rows include enough info that the employee never has to switch screens to figure out which record they're referencing.
- **Reason** — free-text, required. The submission is blocked if empty.

Once the user picks an attendance and types a reason, they tap **Submit**.

---

5. The submit flow

1. Validate that an attendance is selected. If not, toast: *"Please select a late attendance record"*.
2. Validate the reason is non-empty after trim. If empty, toast: *"Please enter a reason for the waiver"*.
3. Show a confirmation alert summarising the request:

   ```
   Date: 26 May 2026
   Late: 18 minutes
   Deduction: ₹150

   Reason: Train delayed by 20 minutes due to signal failure at City Junction
   ```

   Buttons: **Submit** / **Cancel**.
4. On **Submit**:
   - Loading spinner.
   - Call `submitWaiverRequest(empId, attendanceId, reasonStr)`.
   - On success: toast *"Waiver request submitted for approval!"*, clear the form, refresh both the eligible dropdown (so the just-waived record disappears from it) and the My Requests history list, then switch to the **History** tab.
   - On failure: alert with the server's error message.

Submissions go through the offline queue if the device is offline.

---

6. My Requests — the history tab

The Waiver screen has a tab strip:

- **Form** — submit a new waiver.
- **History** — see every waiver you've submitted.

Each history row shows:

- The date of the late check-in being waived.
- Lateness duration (e.g. *"18 min late"*).
- Deduction amount that was on the line.
- Current state: **Pending**, **Approved**, or **Rejected** — colour-coded pill.
- A short snippet of the reason.

Tapping a row expands it to show the full reason and the manager's note (which appears once approved or rejected).

The history list polls every 4 seconds while the user is on the tab, so state changes (manager just approved it) appear without manual refresh.

---

7. The auto-refresh behaviour

A subtle but important detail: while the user is in waiver mode, the app sets up a **4-second polling loop** that:

1. Re-fetches `getEligibleLateAttendances(empId)` so newly-late records appear and just-waived records disappear.
2. Re-fetches `getMyWaiverRequests(empId)` so state transitions reflect.
3. On mount, also opportunistically refreshes the cached late-config and slab values via `getLateConfig` + `fetchAndCacheLateSlabs`. Silent if offline.

This is necessary because:

- The user might have done an offline check-in earlier and the sync just completed — the eligible record only just appeared on the server.
- The manager might have just approved one of their waivers — the state needs to flip live.
- Polling is light (just two RPCs) and 4 seconds is a good trade-off between responsiveness and battery.

The polling stops when the user leaves the waiver tab.

---

8. State lifecycle

```
Pending  →  Approved  →  (deduction reversed)
         →  Rejected  →  (deduction stands)
```

- **Pending** — submitted, awaiting manager review. Default state right after submit.
- **Approved** — manager (or HR) accepted. The deduction for that attendance is reversed in the next payroll cycle.
- **Rejected** — manager declined; the manager's note appears in the history.

Cancellation is **not** supported on the mobile app for waiver requests — once submitted, only the manager can act. (This is intentional: a frivolous waiver-then-cancel pattern would defeat the audit purpose.)

---

9. Server-side rules and deduction reversal

When the manager approves a waiver:

- The waiver record's state flips to **Approved**.
- A reverse-deduction record is created (the exact mechanism depends on the payroll integration in your Odoo deployment — typically a negative-amount entry against the same period).
- A confirmation message is posted to the discussion thread.
- The employee gets a notification.

When the manager rejects:

- The waiver state flips to **Rejected** with the manager's reason in the note field.
- The original deduction stays on the attendance.
- The employee is notified.

Server-side guards:

- One pending waiver per attendance — re-submitting while the first is still pending is blocked.
- Waivers older than the deployment-configured grace window (e.g. 30 days) can't be approved automatically — manager intervention required.
- The waived deduction can't exceed the original deduction (no over-reversal).

---

10. How late deductions get computed in the first place

The waiver only makes sense if you know what's being waived. The late-deduction logic:

1. On check-in, the app records `check_in` timestamp.
2. The server (and the local replica in `computeLocalLateInfo`) compares against `office_start_hour + late_threshold_minutes` for that day's session.
3. If the check-in is past that bound, the difference in minutes is the lateness.
4. Lateness is mapped to a deduction:
    - **Fixed mode** — a flat amount per late arrival (e.g. ₹150 regardless of how late).

    - **Slab mode** — tiered (e.g. 1–15 min = ₹50, 16–30 min = ₹150, 31+ min = ₹300). Slabs come from `hr.attendance.late.slab`.

5. Grace allowances: first N lates per month or per day are free of deduction even if technically late.

The waiver targets a specific `hr.attendance.id` — when the deduction was originally computed, that's the record we want to undo.

---

11. Edge cases

- **Already-waived record reappearing in the dropdown** — shouldn't happen normally; the eligible-records query filters out records with an active waiver. If you see it, it's a deployment quirk and re-submitting will likely error with "duplicate waiver".
- **Manager out of office** — the request stays in **Pending** indefinitely until someone acts on it. Some deployments have a delegate-approver fallback.
- **Late-config changed between check-in and waiver submission** — the original deduction amount is the one stored on the attendance record at the moment of check-in; later config changes don't retroactively alter it.
- **Offline waiver submit** — queued and replayed; the eligible-record dropdown might be slightly stale while offline, but the underlying record is fetched fresh on next reconnect.
- **Employee tries to waive a late check-in they did themselves on purpose** — there's no automated check for "abuse"; this is a managerial judgement call. The reason field is meant to encourage honesty.

---

12. Where the data lives (UI → Odoo)

What you see in the app	Where it lives on the server
Waiver request record	`hr.attendance.waiver` (custom model in the HR addon)
Late attendance being waived	`hr.attendance` (the original record)
Lateness in minutes	Computed field on `hr.attendance` (or stored field, depending on deployment)
Deduction amount	Computed or stored field on `hr.attendance.deduction_amount`
Manager approval thread	`mail.thread` messages on the waiver record
Late config (threshold, deduction mode)	`hr.attendance.late.config`
Late deduction slabs	`hr.attendance.late.slab`

13. Buttons and gates — quick cheat sheet

Button	Visible when
Form tab	Always (default tab on entry).
History tab	Always; auto-switches here after a successful submission.
Attendance dropdown	On the Form tab; populated by `getEligibleLateAttendances`. Empty state shows *"No eligible late records to waive"*.
Reason field	On the Form tab. Required.
Submit	On the Form tab; tap-able after both the attendance and reason are filled.
Cancel on a row	Not supported — waivers can't be cancelled from the mobile app.

14. A worked example

8:15 AM. Priya checks in to the office. Her shift starts at 08:00 with a 15-minute threshold, so 08:15 is *exactly* on the boundary. The local late-info computes the lateness as 0 minutes — no deduction.

The next day she's at 08:22 — that's 22 minutes past 08:00, but with the 15-minute threshold, she's 7 minutes "officially late". The slab table says 1–15 min = ₹50. Her attendance record is saved with `late_minutes = 7, deduction_amount = 50`.

She had a reason: her child's school bus was running late and she dropped him off. She wants to apply for a waiver.

1. Opens the app → **Late Waiver Request**.
2. Verifies with fingerprint.
3. The Form tab is open. The dropdown shows: *"26 May 2026 · 7 min late · ₹50"*. She picks it.
4. Types the reason: *"School bus delay — had to drop child at school personally."*
5. Taps **Submit**.
6. Alert confirms the summary. She taps Submit again.
7. Toast: *"Waiver request submitted for approval!"* The form clears, screen switches to History, her new request shows up at the top in orange **Pending**.
8. Her manager opens Odoo within an hour, reads the reason, clicks Approve and adds a note: *"OK — first time this month, understandable."*
9. Priya gets an email. Next time she opens the app and looks at History, the same row is now green **Approved** with the manager's note visible when she taps the row to expand.
10. In the next payroll cycle the ₹50 deduction is reversed.

That's the full Late Waiver Request cycle.